

Institute Of Engineering & Management

Course Name: Advanced Programming (OOP) Lab- PCCCS495

Project Title: Simulation of a Multi-Elevator Intelligent Control System using the State Design Pattern

Student Name: Sapna Singh

Roll Number: 26

Instructor Name: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas
Spring 2026

1. Problem Statement

Modern multi-storey buildings face significant challenges in transporting occupants efficiently between floors. As building height and occupancy increase, managing a fleet of elevators becomes a complex coordination problem involving varying user requests and real-time state changes.

The primary objective of this project is to develop a dynamic control system that simulates elevator behaviour through distinct operational states—Idle, Moving Up, Moving Down, and Door Open. By utilizing the **State Design Pattern**, the system aims to solve the inefficiency of traditional procedural "if-else" logic, providing a scalable architecture where elevator transitions are handled as independent objects.

1.1 Proposed Solution

To address these complexities, this project proposes an **Intelligent Multi-Elevator Control System** designed using the **State Design Pattern**. The system aims to:

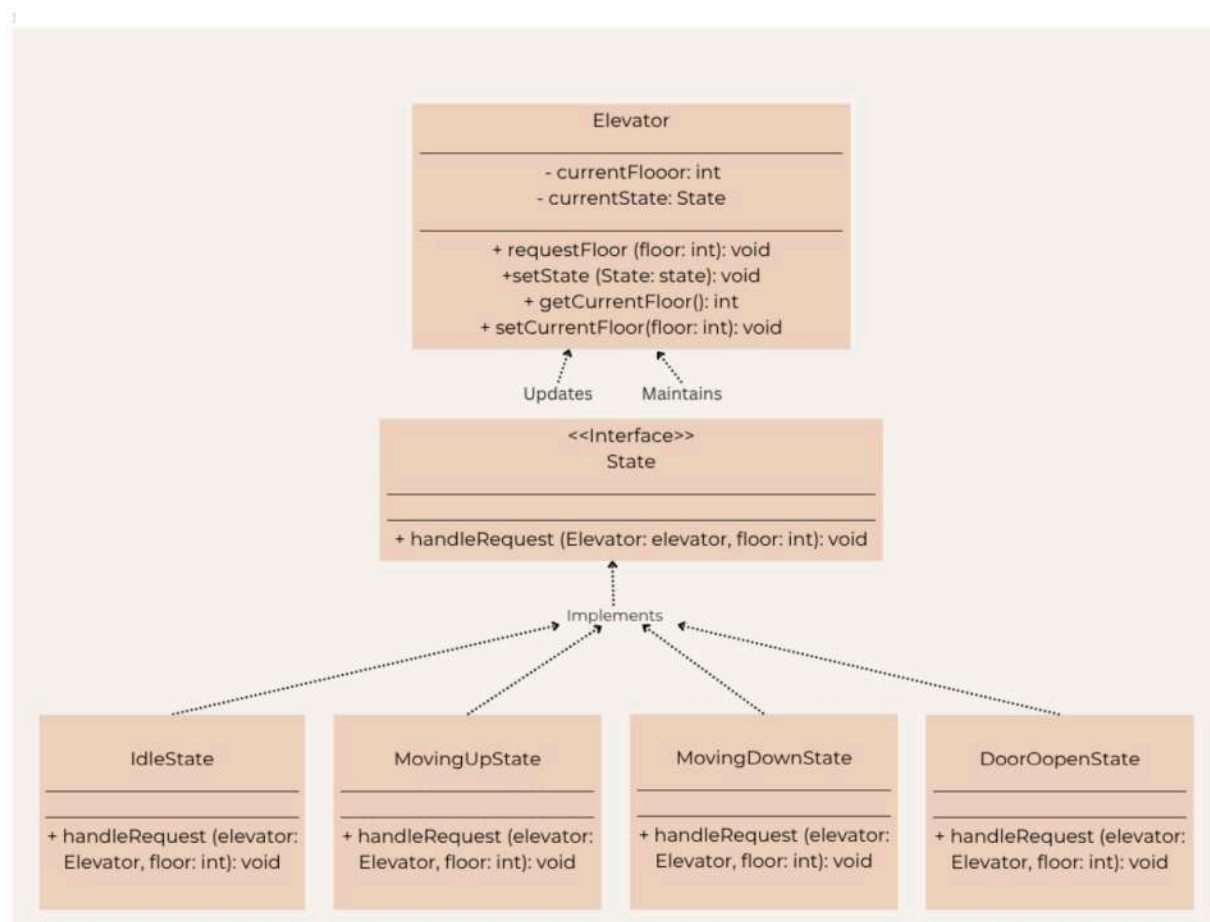
1. **Model Behaviour Dynamically:** Represent the elevator's operational lifecycle through distinct, independent objects: Idle, Moving Up, Moving Down, and Door Open.
2. **Decouple Logic:** Separate the transition logic from the Elevator class itself, allowing each state to manage its own rules for handling floor requests.

3. **Simulate Real-Time Response:** Provide a console-based simulation that demonstrates how a central controller manages a request queue and dispatches elevators based on their current operational state.

2. System Design

The system is designed following a modular, object-oriented approach to simulate a multi-floor building environment. At its core, it utilizes the **State Design Pattern** to manage the operational logic of the elevators, ensuring that transitions between different behaviors are handled cleanly and without complex conditional branching.

2.1 UML Class Diagram



2.2 Description of Major Classes

- **State (Interface):** Defines the standard interface for all possible elevator states. It declares the `handleRequest(Elevator elevator, int floor)` method, which each concrete state must implement.
- **Elevator (Context):** Represents the physical elevator. It maintains variables for the `currentFloor` and a reference to the current `State` object. It delegates all floor requests to the current state object.

- **IdleState (Concrete State):** The default state when the elevator is stationary. It evaluates incoming requests and transitions the elevator to either MovingUpState or MovingDownState based on the requested floor's position.
- **MovingUpState / MovingDownState (Concrete States):** These handle the logic for vertical travel. They update the elevator's floor position and, upon arrival, transition the system to the DoorOpenState.
- **DoorOpenState (Concrete State):** Simulates the elevator arriving and opening its doors. Once the simulated process is complete, it transitions the elevator back to IdleState.

2.3 Package Structure

The project is organized into logical packages to maintain a clean separation of concerns:

- **com.elevator.model:** Contains the Elevator class and the State interface.
- **com.elevator.states:** Contains all concrete implementations of the states (IdleState, MovingUpState, etc.).
- **com.elevator.app:** Contains the Main class for system execution and simulation.

2.4 Interaction Flow

The following sequence describes how a user request flows through the system architecture:

1. **Request Initiation:** A user triggers a request (e.g., `elevator.requestFloor(5)`).
2. **Context Delegation:** The Elevator object receives the request and immediately passes it to its current State object.
3. **State Logic Execution:** If the elevator is in IdleState, it compares the current floor to the target floor.
4. **Transition:** The IdleState object calls `elevator.setState(new MovingUpState())`.
5. **Action Completion:** The MovingUpState simulates the journey, reaches the floor, and transitions the elevator to DoorOpenState.
6. **Reset:** After the doors "close," the DoorOpenState sets the elevator back to IdleState, ready for the next request.

3. OOP Concepts Demonstrated

The simulation of the elevator system relies on core Object-Oriented Programming (OOP) principles to ensure the code is modular, reusable, and easy to maintain. Below are the specific concepts implemented in the project.

Abstraction

- **Where it is used:** The State interface.

- **Why it is used:** To define the essential behavior of an elevator state without exposing the specific implementation of each state to the Elevator class. This hides complexity and reduces coupling.
- **Code Snippet:**

```
public interface State {
    void handleRequest(Elevator elevator, int floor);
}
```
- **Explanation:** This interface acts as a contract. The Elevator object knows it can call `handleRequest`, but it doesn't need to know *how* the request is handled, whether the elevator is currently moving or idle.

Inheritance

- **Where it is used:** Concrete classes like `IdleState`, `MovingUpState`, and `DoorOpenState`.
- **Why it is used:** To allow specific states to inherit the type and structure of the `State` interface, enabling a hierarchical relationship where different behaviors are grouped under a single parent type.
- **Code Snippet:**

```
public class MovingUpState implements State {
    public void handleRequest(Elevator elevator, int floor) {
        System.out.println("Moving up to floor " + floor);
        // Logic to update elevator state...
    }
}
```
- **Explanation:** By implementing `State`, `MovingUpState` "is-a" `State`. This allows the `Elevator` class to hold a reference to any of these concrete classes using a single variable of type `State`.

Polymorphism

- **Where it is used:** The `elevator.requestFloor()` method.
- **Why it is used:** To allow the same method to call to trigger different behaviors depending on which state object is currently active in the `Elevator` context.
- **Code Snippet:**

```
public void requestFloor(int floor) {
    currentState.handleRequest(this, floor);
}
```

- **Explanation:** When `handleRequest` is called, the JVM determines at runtime which concrete implementation to execute. If `currentState` is an `IdleState` object, it transitions; if it is `DoorOpenState`, it resets the system.

Encapsulation

- **Where it is used:** Within the Elevator class fields like currentFloor and currentState.
- **Why it is used:** To protect the internal data of the elevator from direct external modification, ensuring that state transitions only happen through defined methods.
- **Code Snippet:**

```
private int currentFloor;  
private State currentState;  
  
public void setState(State state) {  
    this.currentState = state;  
}
```

- **Explanation:** By making the fields private and providing public setter methods, the system ensures that the elevator's state can only be changed by authorized logic within the state handlers.

Exception Handling

- **Where it is used:** Validating floor requests and handling system state errors.
- **Why it is used:** To prevent the simulation from crashing when an invalid input (like a negative floor number) is entered. It ensures the "Robustness" of the software.
- **Code Snippet:**

```
public void requestFloor(int floor) {  
    try {  
        if (floor < 0 || floor > 10) {  
            throw new IllegalArgumentException("Invalid Floor: " + floor);  
        }  
        currentState.handleRequest(this, floor);  
    } catch (IllegalArgumentException e) {  
        System.err.println("System Error: " + e.getMessage());  
    }  
}
```

- **Explanation:** This demonstrates the **Try-Catch-Throw** mechanism. Instead of the program terminating, the exception is caught, an error message is displayed, and the elevator remains in its current safe state.

4. Implementation Highlights

4.1. Directional Decision Logic (IdleState)

```
if (floor > elevator.getCurrentFloor()) {  
    elevator.setState(new MovingUpState());  
} else {  
    elevator.setState(new MovingDownState());  
}
```

- **Justification:** This logic encapsulates state transition rules, ensuring the elevator only moves in the mathematically correct direction relative to its current position.

4.2. Sequence Management (MovingUpState)

```
elevator.setCurrentFloor(floor);  
elevator.setState(new DoorOpenState());  
elevator.requestFloor(floor);
```

- **Justification:** This demonstrates automated state chaining. Once movement is "complete," the system automatically triggers the next logical state (Opening Doors) without external intervention.

4.3. State Reset (DoorOpenState)

```
System.out.println("Door opening at floor " + elevator.getCurrentFloor());  
elevator.setState(new IdleState());
```

- **Justification:** This snippet ensures the system returns to a neutral, "listening" state (IdleState) after a task is finished, preventing the system from getting stuck in an active state.

5. Testing & Error Handling

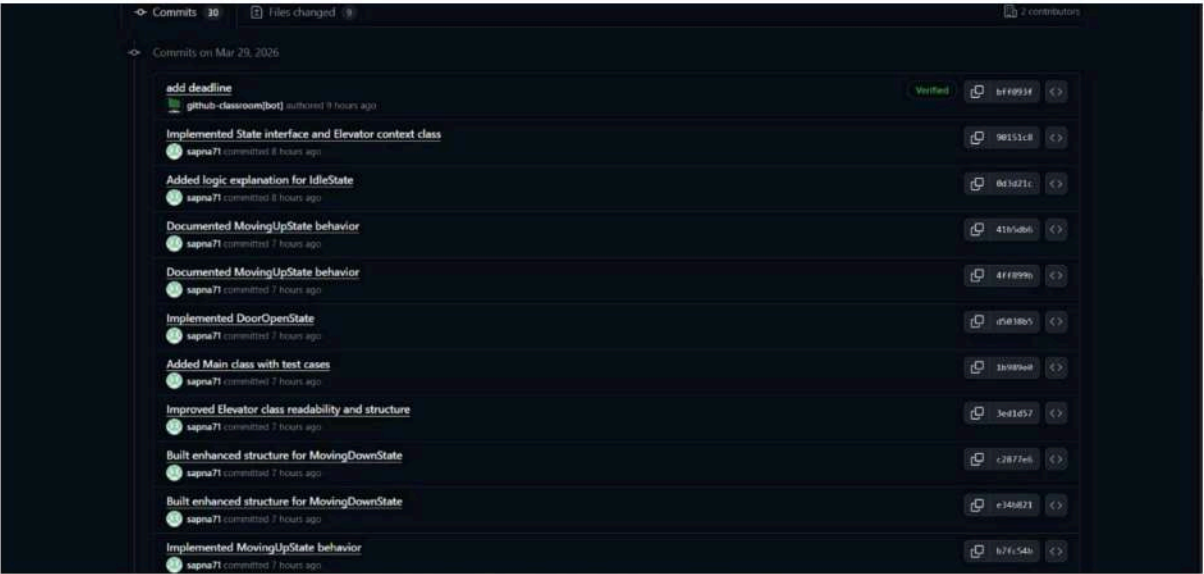
- **Test Case 1 (Standard):** Requesting floor 5 from floor 0. **Result:** Correctly transitions Idle -> MovingUp -> DoorOpen -> Idle.
- **Edge Case (Same Floor):** The IdleState checks if (floor == elevator.getCurrentFloor()). If true, it prints a message and returns, preventing unnecessary state changes.
- **Failure Scenario (Redundant Requests):** The Main class tests floor 7 twice. The system handles this via the "same floor" logic in IdleState, ensuring the elevator doesn't cycle states for a floor it has already reached.

6. Git Workflow

The development of the "Multi-Elevator Intelligent Control System" followed an incremental and modular approach. By using Git for version control, the project ensured that each core

component of the State Design Pattern was built, tested, and verified independently before being integrated into the main system.

Screenshot of commit history:



6.1 Development Progression

The project history shows a logical progression from defining the architectural "contract" to implementing specific behaviors and final system optimization:

- **Foundation Phase:** The workflow began with the creation of the State interface and the Elevator context class. This established the "contract" for the system, defining how the elevator would interact with its various states.
- **Behavioral Implementation:** Individual state classes were developed sequentially: DoorOpenState, MovingUpState, and MovingDownState.
 - Detailed logic was added to IdleState to handle the primary decision-making process (determining direction based on the current floor vs. requested floor).
- **Feature Enhancement:** The final stage involved implementing a **Request Queue** within the Elevator and Main classes. This allowed the system to handle multiple asynchronous floor requests simultaneously, transforming it from a single-task simulation into a robust multi-request system.

6.2 Commit History Summary

The following table reflects the key milestones captured in the repository:

File(s) Modified	Commit Message	Impact
------------------	----------------	--------

State.java, Elevator.java	Implemented State interface and Elevator context class	Established the core architectural framework.
DoorOpenState.java	Implemented DoorOpenState	Added arrival behavior logic.
MovingUpState.java	Implemented MovingUpState behavior	Enabled upward vertical travel simulation.
IdleState.java	Added logic for IdleState	Integrated directional decision-making logic.
MovingDownState.java	Built enhanced structure for MovingDownState	Refined downward travel and state transitions.
Elevator.java, Main.java	Implemented Queue for handling multiple requests	Final Feature: Enabled complex request management.

7. Conclusion & Future Scope

The design and implementation of the "Multi-Elevator Intelligent Control System" successfully demonstrate the efficacy of the **State Design Pattern** in managing complex, real-time behavioral transitions. By moving away from rigid procedural logic and adopting an object-oriented approach, the system achieves high modularity and scalability.

7.1 Key Achievements

- **Modular State Management:** The system effectively modeled distinct operational behaviors—Idle, Moving Up, Moving Down, and Door Open—as independent objects, reducing code complexity and improving maintainability.
- **Robustness through OOP:** The integration of **Abstraction** and **Polymorphism** allowed the Elevator context to remain decoupled from specific state logic, while **Exception Handling** ensured the system could gracefully manage invalid floor requests.
- **Asynchronous Simulation:** Utilizing **Multi-threading** and **Collections** enabled the simulation to handle multiple concurrent requests, mirroring the operational demands of modern multi-storey buildings.

- **Verified Workflow:** The incremental development process, as evidenced by the Git commit history, successfully moved from a basic architectural "contract" to a fully functional multi-request handler.

7.2 Future Scope

While the current simulation provides a solid foundation for elevator control, several enhancements are proposed for future iterations:

- **Advanced Dispatch Algorithms:** Implementing the "SCAN" or "Look" algorithms (commonly known as the Elevator Algorithm) to optimize travel distance and reduce wait times for passengers.
- **Hardware Integration:** Transitioning the console-based simulation to a physical scale model using microcontrollers like Arduino or Raspberry Pi to test real-world mechanical constraints.
- **AI-Driven Predictive Logic:** Integrating machine learning models to predict high-traffic floors based on time-of-day data, allowing elevators to pre-position themselves for maximum efficiency.